

3 | Codierung

Wenn wir ein Foto anschauen, sehen wir ein Bild. Wenn wir ein Lied hören, hören wir Musik. Ein Computer nimmt diese Dinge jedoch nicht so wahr wie wir. Er hat ja keine Augen oder Ohren. Für ihn besteht ein Foto, ein Lied oder ein Video aus einer langen Folge von 0 und 1, weil das ist ja seine Sprache. Jede einzelne 0 oder 1 ist dabei ein **Bit**.

Damit wir aber nicht solch für unsere Augen nichts aussagenden Ansammlungen von 0 und 1 anschauen müssen, übersetzt der Computer diese binären Anordnungen in Pixel oder Buchstaben. Diese Übersetzung zwischen menschenverständlichem und computerverständlichem bezeichnet man als **Codierung** und **Decodierung**.



1. Textcodierung

Bei der Textcodierung geht es also darum, wie der Computer Text (Buchstaben, Emojis, ...) speichern, darstellen und/oder übertragen möchte. Bevor wir unsere heutigen Computer hatten, gab es schon erste Technologien, die fähig waren, Buchstaben und Zahlen über Ferne zu übertragen. Schauen wir uns mal folgendes Beispiel an:

Das Fernschreibgerät mit Telex



Das Gerät das Sie links sehen ist ein Fernschreibgerät, das mit dem Telex System mittels elektrischer Signale weltweit Nachrichten austauschen konnte. Quasi ein Vorgänger der schriftlichen Kommunikation, wie wir sie heute auf unseren Smartphones kennen. Unten an der Tastatur konnte der Benutzer eine Nachricht schreiben, welche dann an einen anderen Benutzer gesendet werden konnte. Der konnte dann die Nachricht ausgedruckt auf seinem Stück Papier sehen. Aber was passierte zwischen Schreiben und Drucken im Hintergrund?

Um diese Abfolge zwischen geschriebenem Buchstaben, Übertragung und ausgedrucktem Buchstaben zu verstehen, betrachten Sie die Abbildung 1.

Die Tabelle zeigt, wie die einzelnen Zeichen codiert wurden. Schwarz bedeutet, dass Strom fließt, Weiss bedeutet, dass kein Strom fließt. Wenn Sie beispielsweise den Buchstaben A eingeben, sendet der Fernschreiber zunächst ein Startimpuls. Dieses signalisiert dem empfangenden Gerät, dass nun ein neues Zeichen übertragen wird. Anschliessend werden fünf Bits übertragen. Beim A lautet die Folge:

Strom – Strom – kein Strom – kein Strom – kein Strom

Zum Abschluss folgt ein Stoppimpuls, das das Ende des Zeichens signalisiert.

Der empfangende Fernschreiber liest diese Folge und vergleicht sie mit derselben Codetabelle. Dadurch erkennt er, dass die übertragene Zeichenfolge für den Buchstaben A steht, und kann diesen ausdrucken. Danach beginnt die Übertragung des nächsten Zeichens.

Dieses Verfahren wird als Start-Stopp-Verfahren bezeichnet. Die Buchstaben wurden also nicht direkt übertragen, sondern zunächst in eine Folge von zwei möglichen Zuständen (Strom oder kein Strom) **codiert**.

Aufgabe 1

Für einen Buchstaben gibt es 1 Startimpuls, 5 Impulse (5 Bit) für das Zeichen und 1.5 Stoppimpulse (eigentlich kein Bit, sondern eine Zeit). Das Ganze dauert 150ms. Wie lange dauert ein einzelner Impuls? Wie viele Zeichen pro Minute sind möglich?

NR. VAN DE COMBINATIE	LETTERS	CIJFERS	NR. VAN DE ELEMENTEN					
			START	1	2	3	4	5
1	A	—						
2	B	?						
3	C	:						
4	D							
5	E	3						
6	F							
7	G							
8	H							
9	I	8						
10	J	DEL						
11	K	(						
12	L	)						
13	M	.						
14	N	*						
15	O	9						
16	P	0						
17	Q	!						
18	R	4						
19	S	'						
20	T	5						
21	U	7						
22	V	=						
23	W	2						
24	X	/						
25	Y	6						
26	Z	+						
27	TERUGLOOP WAGEN							
28	NIEUWE REEL							
29	LETTERS							
30	CIJFERS							
31	TUSSENRUIMTE							
32	5 x STROOMLOOS							

Abbildung 1: Der Telex Code

Der Morsecode

Ein anderes System, um Texte über weite Strecken zu senden ist der Morsecode. Jeder Buchstabe wird durch eine Folge aus Punkten und Strichen dargestellt. Ein Punkt steht für ein kurzes Signal, ein Strich für ein langes Signal (siehe die Beispielbuchstaben in Abbildung 2). Die Zeichen sind unterschiedlich lang, ein E besteht beispielsweise aus einem einzelnen Punkt und das J aus einem Punkt und drei Strichen.

A — • —	G — • •	U • • —
B — • • •	H • • • •	V • • • —
C — • — •	I • •	W • — —
D — • •	J • — —	X • — • —
E •	K • — •	Y • — —
F • • — •	L • — • •	Z — — • •

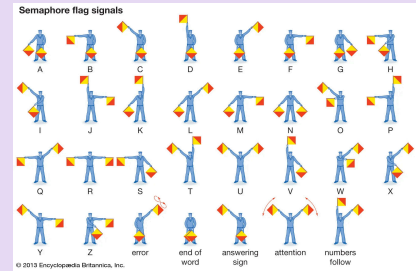
Abbildung 2: Morsecode

Andere historische Textcodierungen

Das Telex System oder der Morsecode waren nicht die einzigen, die es Menschen ermöglichte, Texte über weite Strecken zu senden. Der **Semaphor Code** wurde zum Beispiel in der Schifffahrt verwendet, um zwischen Booten über weitere Distanzen zu kommunizieren. Die Buchstaben werden durch unterschiedliche Positionen von zwei Flaggen dargestellt. Jede Stellung der Arme entspricht einem bestimmten Buchstaben (siehe rechts).



Ein ähnliches Prinzip wurde auch bei den optischen Telegrafen verwendet. In Frankreich entstand Ende des 18. Jahrhunderts ein Netz aus Telegrafentürmen, die über grosse Distanzen miteinander verbunden waren. Auf jedem Turm wurden bewegliche Signalarme in verschiedene Positionen gebracht. Die nächste Station beobachtete das Signal und gab es an die nächste Station weiter. Auf diese Weise konnten Nachrichten über grosse Entfernungen übertragen werden.



Keine grosse Überraschung, aber sowohl der Telex sowie auch der Morsecode sind heute von modernen Technologien überholt worden. Die Frage ist aber, warum war man mit diesen Lösungen nicht zufrieden?

Aufgabe 2

Telex und Morsecode ermöglichten die Übertragung von Nachrichten über grosse Distanzen. Doch beide Verfahren haben ihre Schwachstellen. Versuchen wir sie herauszufinden.

Beim Telex wird jedes Zeichen mit 5 Bits codiert und beim Morsecode sind die Zeichen unterschiedlich lang.

- Wie viele verschiedene Zeichen könne mit 5 Bits grundsätzlich dargestellt werden?
- Reicht diese Anzahl aus, um alle Zeichen darzustellen, die wir heute beim Schreiben benötigen?
- Sie möchten die Nachricht «GE» übertragen. Schreiben Sie die entsprechende Folge aus Punkten und Strichen auf.
- Könnte ein Empfänger anhand dieser Folge eindeutig die einzelnen Zeichen erkennen? Was müsste noch zusätzlich gesendet werden, damit die Eindeutigkeit gewährleistet ist?
- Welche Eigenschaften müsste eine neue Zeichencodierung besitzen, damit sie die Schwachstellen von Telex und Morsecode möglichst gut löst?

1.1. ASCII Codierung

1963 wurde schliesslich der **ASCII-Code** eingeführt. Dieser verzichtete auf Start- und Stoppbits und wurde ursprünglich für alle Zeichen einheitlich auf 7 Bit festgelegt.

Diese Zeichen beinhalten:

- 33 nicht druckbare Steuerzeichen
- 95 echte Zeichen (siehe rechts)

	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	{		}	~	

Diese Codes für jedes einzelne Zeichen müssen Sie nicht auswendig lernen. Für das gibt es ASCII-Tabellen, welche Sie auch an der Prüfung benutzen dürfen. Schauen wir uns eine solche Tabelle konkret an ein paar Beispielen an:

ASCII-Tabelle lesen

In der Tabelle sehen Sie, dass dem Buchstaben A der Wert 65 zugeordnet ist. Diese Zuordnung wurde festgelegt und jedes Zeichen erhält dadurch einen eindeutigen Code. Das kleine a hat zum Beispiel den Wert 97. Das sind Dezimalzahlen. Der Computer speichert die Informationen jedoch nicht als Dezimalzahlen, sondern binär.

Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
38	26	046	&	&	70	46	106	F	F	102	66	146	f	f

In der Tabelle finden Sie deshalb auch den hexadezimalen Wert. Die Hexadezimalschreibweise ist kompakter und benötigt weniger Platz als die vollständige binäre Darstellung. Beim Buchstaben A ist der hexadezimale Wert beispielsweise 41₁₆. In Binärschreibweise entspricht dies: 100'0001₂ (7 Bit!).

5	5	005	ENQ	{enquiry}
6	6	006	ACK	{acknowledge}
7	7	007	BEL	{bell}
8	8	010	BS	{backspace}
9	9	011	TAB	{horizontal tab}
10	A	012	LF	{NL line feed, new line}

Die sichtbaren Zeichen beginnen in der Tabelle erst ab dem Wert 32. Davor befinden sich sogenannte Steuerzeichen. Diese lösen bestimmte Aktionen aus. Einige davon sind Ihnen wahrscheinlich bereits bekannt: Mit backspace löschen Sie beispielsweise ein Zeichen, und mit new line erzeugen Sie einen Zeilenumbruch.

Für die folgenden Aufgaben brauchen Sie eine ASCII-Tabelle, die Sie direkt unterhalb einsehen können.

Aufgabe 3

Schreiben Sie den folgenden Satz in ASCII, indem Sie die Codes in **dezimaler** Schreibweise hinschreiben:

Macht Informatik Spass?

Aufgabe 4

Schreiben Sie den folgenden Satz in ASCII, indem Sie die Codes in **binärer** Schreibweise hinschreiben:

Heute ist es schön 😊

ASCII-Tabelle

Dec	Hex	Character
0	00	NUL
1	01	SOH
2	02	STX
3	03	ETX
4	04	EOT
5	05	ENQ
6	06	ACK
7	07	BEL
8	08	BS
9	09	HT
10	0A	LF
11	0B	VT
12	0C	FF
13	0D	CR
14	0E	SO
15	0F	SI
16	10	DLE
17	11	DC1
18	12	DC2
19	13	DC3
20	14	DC4
21	15	NAK
22	16	SYN
23	17	ETB
24	18	CAN
25	19	EM
26	1A	SUB
27	1B	ESC
28	1C	FS
29	1D	GS
30	1E	RS
31	1F	US
32	20	(Space)
33	21	!
34	22	"
35	23	#
36	24	\$
37	25	%
38	26	&
39	27	'
40	28	(
41	29	)
42	2A	*
43	2B	+
44	2C	,
45	2D	-
46	2E	.

Dec	Hex	Character
47	2F	/
48	30	0
49	31	1
50	32	2
51	33	3
52	34	4
53	35	5
54	36	6
55	37	7
56	38	8
57	39	9
58	3A	:
59	3B	;
60	3C	<
61	3D	=
62	3E	>
63	3F	?
64	40	@
65	41	A
66	42	B
67	43	C
68	44	D
69	45	E
70	46	F
71	47	G
72	48	H
73	49	I
74	4A	J
75	4B	K
76	4C	L
77	4D	M
78	4E	N
79	4F	O
80	50	P
81	51	Q
82	52	R
83	53	S
84	54	T
85	55	U
86	56	V
87	57	W
88	58	X
89	59	Y
90	5A	Z
91	5B	[
92	5C	\
93	5D	]

Dec	Hex	Character
94	5E	^
95	5F	_
96	60	`
97	61	a
98	62	b
99	63	c
100	64	d
101	65	e
102	66	f
103	67	g
104	68	h
105	69	i
106	6A	j
107	6B	k
108	6C	l
109	6D	m
110	6E	n
111	6F	o
112	70	p
113	71	q
114	72	r
115	73	s
116	74	t
117	75	u
118	76	v
119	77	w
120	78	x
121	79	y
122	7A	z
123	7B	{
124	7C	
125	7D	}
126	7E	~
127	7F	DEL

ASCII scheint also ebenfalls nicht ganz vollständig unsere modernen Anforderungen zu erfüllen. Zwei der Zeichen aus den Sätzen von den Aufgaben sind nicht in der Tabelle enthalten... Haben Sie eine Ahnung, warum Zeichen mit Akzent (é, â, ö, ñ) nicht enthalten sind? Die Antwort liegt im A von ASCII.

1.2. Weitere Codierungen

Eine naheliegende Idee war, ASCII zu erweitern: Statt 7 Bits verwendet man 8 Bits. Genau das machte IBM beispielsweise mit **Code Page 437**. Die ersten 128 Zeichen blieben unverändert und entsprachen ASCII. Die zusätzlichen 128 Zeichen wurden mit weiteren Buchstaben, Symbolen und grafischen Zeichen gefüllt.

Aufgabe 5

Wie viele verschiedene Zeichen können mit 8 Bits dargestellt werden? Wie viele davon kommen im Vergleich zu 7 Bits hinzu?

Wenn wir uns die Code Page 437 in Abbildung 4 genauer betrachten, können wir schnell überprüfen, dass die Zeichen für die ASCII Werte dieselben sind wie vorher:

Nehmen wir mal den Grossbuchstaben A. In der Tabelle sehen wir, dass er in Zeile 4, Spalte 1 ist. Das kann man so lesen, dass der Hexadezimalwert von A 41₁₆ ist. Das ist derselbe Wert, den A in der Tabelle in Abbildung 3 hat.

Nun konnte man den Computer auch in Ländern mit Sprachen mit Umlauten (z.B. Schweiz) verkaufen. Ein paar der Zeichen sehen aber sehr speziell aus. Diese Linien, die man z.B. in der Zeile C sieht, dienten damals zur Hilfe, um grafisch zu malen. So sehen Sie in Abbildung 5 die rot eingezeichneten Quadrate entsprechen den Zeichen B3, C1 und C7 in der Code Page 437.

Aufgabe 6

Ein Problem bleibt aber. 128 zusätzliche Zeichen reichen immer noch nicht für alle Sprachen auf der Welt. Kennen Sie ein Alphabet wessen Zeichen nicht in dieser Code Page 437 enthalten sind?

Deshalb wurden weitere Codepages entwickelt. Codepage 850 war beispielsweise stärker auf westeuropäische Sprachen ausgerichtet und enthielt dafür andere Zeichen als Codepage 437 (wobei die ursprünglichen ASCII-Zeichen dieselben blieben).

Code Page 437

	-0	-1	-2	-3	-4	-5	-6	-7	-8	-9	-A	-B	-C	-D	-E	-F
0-	*	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣
1-	1	2	3	4	5	6	7	8	9	:	<	=	>	?	␣	␣
2-	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/	␣
3-	0	1	2	3	4	5	6	7	8	9	:	<	=	>	?	␣
4-	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	␣
5-	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	␣
6-	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	␣
7-	p	q	r	s	t	u	v	w	x	y	z	{		}	~	␣
8-	ç	ü	é	â	ä	å	ä	å	ö	ø	ï	î	ï	ä	å	␣
9-	ê	æ	à	ô	ö	ù	û	ü	ø	û	ç	£	¥	℞	ƒ	␣
A-	á	í	ó	ú	ñ	ã	ẽ	ẽ	ẽ	ẽ	ẽ	ẽ	ẽ	ẽ	ẽ	␣
B-	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣
C-	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣
D-	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣
E-	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣
F-	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣

Used first in IBM Monochrome Display Adapter in 1981.

Abbildung 4: Code Page 437, wie sie IBM 1981 darstellte. Die Reihen und Spalten sind Hexadezimalzahlen.

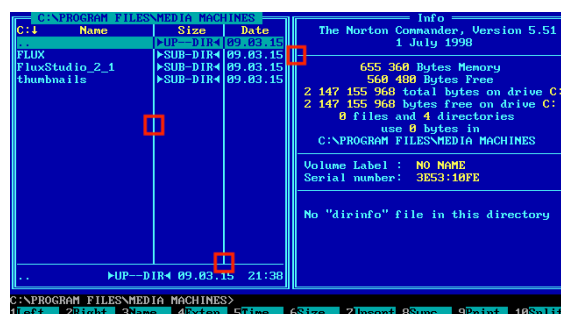


Abbildung 5: Der Norton Commander war eine frühere Version ihres heutigen Explorers/Finders auf ihrem Windows/Mac.

437	Die ursprüngliche Zeichensatztable des IBM-PC	852	Slawische Sprachen (Latin-2), zentraleuropäische und osteuropäische Sprachen	862	Hebräisches Alphabet
720	Arabisches Alphabet	855	Kyrrilisches Alphabet	863	Lateinisches Alphabet mit französischen Sonderzeichen
737	Griechisches Alphabet	857	Türkisches Alphabet	864	Arabisches Alphabet
775	Estrnisches Alphabet, Litauisches Alphabet und Lettisches Alphabet	858	„Multilingual“ mit Eurozeichen	865	Dänisch und Norwegisch – unterscheidet sich von 437 nur durch Ø (ø) anstelle von ¥ und ¢
819	„Latin-1“, entspricht ISO 8859-1	860	Lateinisches Alphabet mit portugiesischen Sonderzeichen	866	Kyrrilisches Alphabet
850	„Multilingual (DOS-Latin-1)“, westeuropäische Sprachen	861	Isländisches Alphabet	869	Griechisches Alphabet

Abbildung 6: Beispiele verschiedener Codepages. Sie finden noch einige mehr unter <https://de.wikipedia.org/wiki/Zeichensatztable>

Jetzt stellen Sie sich nun aber mal vor, Sie schreiben einen Text auf ihrem Computer in der Schweiz (Codepage 850). Anschliessend schicken Sie diesen Text an jemanden, dessen Computer griechisch eingestellt ist (Code Page 869). Die hexadezimalen Zahlen werden zwar korrekt übertragen – aber der Empfänger interpretiert sie mit einer anderen Tabelle. Aus einem Zeichen kann dadurch plötzlich ein völlig anderes Zeichen werden.

Diese falsche Interpretation können Sie in Abbildung 7 sehen: Die Linien, welche in Abbildung 5 noch schön dargestellt wurden, entsprechen jetzt zum Teil dem Zeichen ä.

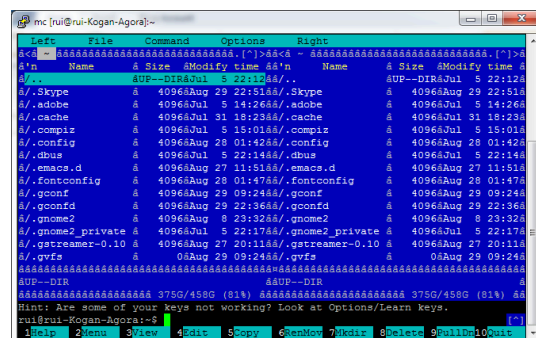


Abbildung 7: Der Norton Commander mit einer falschen Darstellung der Linien.

1.3. Unicode

Die vielen verschiedenen Codepages führten zu einem grossen Problem: Es gab zwar immer mehr Zeichen, aber nicht überall dieselben. Ein Zeichen konnte auf einem Computer etwas anderes bedeuten als auf einem anderen. Es brauchte deshalb eine einheitliche Zeichencodierung, die möglichst alle Zeichen der Welt umfasst.

Die Idee dahinter ist zunächst einfach: Wenn wir mehr Bits verwenden, können wir viel mehr verschiedene Zeichen darstellen. Das sieht man ganz schnell, wenn man sich die Auswirkungen der Zweierpotenzen anschaut:

- $2^7 = 128$ mögliche Zeichen (ASCII) bei 7 Bit
- $2^8 = 256$ mögliche Zeichen (Codepage) für 1 Byte
- $2^{16} = 65'536$ mögliche Zeichen bei 2 Bytes
- $2^{32} = 4'294'967'296$ mögliche Zeichen bei 4 Bytes

Genau hier setzt **Unicode** an. Unicode ist ein weltweit einheitliches System, bei dem Zeichen aus verschiedensten Sprachen eine eindeutige Nummer erhalten. Dazu gehören beispielsweise lateinische, griechische und chinesische Schriftzeichen, aber auch mathematische Symbole, Sonderzeichen und Emojis (siehe).

Diese vielen verschiedenen Codepages waren unglaublich chaotisch. Und wieder das Problem hier war, dass es einfach zu wenig Platz für alle Zeichen gab... Dieses Problem kann man schnell lösen, sobald man mehr Bit hinzufügt. Und so kam, dass wohl oder übel endlich ein einheitliches globales System eingeführt wurde: Unicode (oder UTF-8?). Die Anzahl lässt sich schnell erhöhen. Wenn wir anstatt 1 Byte jetzt 2 Byte in Betracht ziehen, haben wir schon $2^{16} = 65'536$ mögliche Zeichen! Bei 4 Byte sind es schon $2^{32} = 4'294'967'296$!

Der Unicode enthält ungefähr 150'000 Zeichen aus verschiedensten Sprachen der Welt. Er enthält aber auch Emojis und weitere Zeichen. Es kommen regelmässig neue dazu. Davon haben Sie sicher gehört, dass es auf einmal heisst, neue Emojis werden bald auf ihr Handy kommen. Der Grund, warum man das machen kann, ist diese unglaubliche Anzahl möglicher Zeichen, die man mit 4 Byte darstellen kann. 4 Bytes sind aber auch sehr viel für jedes einzelne Zeichen... stellen Sie sich ein Wort von 5 Zeichen vor, besteht schon aus 20 Bytes.

(Insert ein paar Beispiel: Japanische Zeichen, Griechische, Emoji, Sonderzeichen etc)

Und so kam der UTF: Unicode Transformation Format. Das löste das Speicherproblem, weil nur so viele Bytes verwendet werden, wie für das Zeichen nötig sind. Aber jetzt sind wir ja wieder beim gleichen Problem wie beim Morsecode... Unterschiedliche Länge der Zeichen.. Woher weiss der computer, wann ein zeichen anfängt und wann es endet?

Erklärung mit Beispiel des Zuges. Am Anfang sitzt der Lokführer und der weiss, wie viele Wagen er hat. Und jeder Wagen hat einfach ein Zeichen, dass er dazugehört. Der Lokführer signalisiert mit einer Abfolge, wie viele Wagen der ganze Zug beinhaltet: Wenn der Zug 2 Wagen hat (also 2 Bytes), dann hat das erste Byte (der Lokführer) die Kombination 110 am anfang. Wenn er insgesamt 3 Wagen (3 Bytes) hat, dann beginnt er mit 1110 und bei 4 Wagen mit 11110. Jeder Wagen dahinter beginnt einfach mit 10. Wenn der Zug nur 1 Wagen hat (1 Byte) ist es der gleiche Fall wie bei ASCII und beginnt einfach mit 0 (ASCII war ja 7 Bits lang).

(Insert Beispielbild mit Zug und UTF-8 Kodierung daneben als Tabelle)

In Europa verwendet man den UTF-8 (was ist der Unterschied sonstwo?). Die ersten 128 Zeichen entsprechen der ASCII Zeichen und brauchen auch genau gleich viel Platz wie früher bei reinem ASCII der Fall war. Nur weniger häufige Zeichen benötigen mehr Speicherplatz.

2. Bildercodierung

Lösungen

Lösung 1

$\frac{150}{7.5} \text{ ms} = 20 \text{ ms}$ für die ersten 6 Impulse und 30ms für den Stoppimpuls.

Eine Sekunde hat 1000ms, dh. pro Sekunde können $\frac{1000}{150} \approx 6.67$ Zeichen geschrieben werden. Pro Minute bedeutet das 400 Zeichen. Das ist nicht die schnellste Technologie aber sicher auch nicht so langsam, wie man sich das vielleicht vorstellt!

det wird, ist nicht in diesen Zeichen enthalten. Ein anderes Beispiel wäre auch das griechische Alphabet.

Lösung 2

- a) Bei 5 Bit gibt es $2^5 = 32$ Möglichkeiten. Das heisst 32 verschiedene Zeichen sind möglich.
- b) Nein. 32 Zeichen reichen nicht aus, um beispielsweise alle Gross- und Kleinbuchstaben, Zahlen, Satzzeichen und weitere Sonderzeichen (Emojis?) darzustellen.
- c) G (— ·) und E (·) zusammen ergibt — · ·
- d) Nein. Die Folge — · · könnte als GE verstanden werden, aber auch als Buchstabe Z. Der Empfänger weiss ohne zusätzliche Information also nicht, wo das G endet und das E beginnt. Es müsste also eine Pause oder ein zusätzliches Trennsignal zwischen den Zeichen gesendet werden.
- e) Eine geeignete (moderne) Zeichencodierung müsste genügend verschiedene Zeichen darstellen können und eindeutig sein, damit der Empfänger weiss, wo ein Zeichen beginnt und endet.

Lösung 3

77 97 99 104 116 32 73 110 102 111 114 109 97 116
105 107 32 83 112 97 115 115 63

Lösung 4

1001000 1100101 1110101 1110100 1100101 0100000
1101001 1110011 1110100 0100000 1100101 1110011
0100000 1110011 1100011 1101000 1101111 1100101
1101110

Für das ö haben wir kein geeignetes ASCII-Zeichen gesehen, darum haben wir es mit oe ersetzt. Für das Emoji haben wir aber bisher keine Lösung...

Lösung 5

$2^7 = 128$ und $2^8 = 256$. Es kommen also 128 neue Zeichen dazu (doppelt so viele wie vorher).

Lösung 6

Das kyrillische Alphabet, welches z.B. für die ukrainische, bulgarische oder auch russische Sprache verwen-